

HTTP API

The server speaks one wire format for songs: `.sng`, whatever shape a song was found in on disk. That is the design commitment in ADR-001 — an unmodified YARG reads a `.sng` natively, so a client can write ordinary files into an ordinary songs folder and the game needs no change at all.

Everything below is v1 and unversioned only in the sense that `/api/v1` is the version.

This API has a first-party consumer. `yarg-sync` (`cmd/yarg-sync`, documented in `SYNC-CLIENT.md`) is built against it, and `internal/e2e` runs the real client against the real handler on every pipeline. Three things below are load-bearing for it rather than incidental, so changing them breaks a shipped binary:

- **`POST /api/v1/have`** is how the client takes inventory in one round trip, and its empty-list form is how `-prune` learns the server's full set.
- **300 Multiple Choices** on a shared chart hash. The client relies on the server *not* choosing, and resolves it deterministically itself so that two machines syncing the same library pick the same package. (Picking the same package is necessary for a shared library; it is not sufficient. Until 2026-09-06 the two machines picked the same package and still received different bytes, because packing itself was not deterministic.)
- **`ETag` as the package hash, and `Range`.** Identity is re-derived from the received bytes client-side, so a substituted or truncated response fails closed. Both of these rest on packing being deterministic — one `ETag` must mean one sequence of octets, or a resume splices two different archives together. It did not hold until 2026-09-06; see `docs/TEST-CORPUS.md`.

Endpoints

Method	Path	What it is for
`GET`	`/`	The browse page, when `browse_ui` is on.
`GET`	`/healthz`	Liveness. Returns `ok`.
`GET`	`/version`	The build's version string.
`GET`	`/api/v1/library`	What was indexed, and what could not be.
`GET`	`/api/v1/features`	Which optional capabilities are on, and why.
`PUT`	`/api/v1/features/{name}`	Turn one on or off. Off by default; see `config_writes`.

Method	Path	What it is for
`GET`	`/api/v1/songs`	Browse and search.
`GET`	`/api/v1/songs/{chart_hash}`	Every package sharing a chart hash.
`POST`	`/api/v1/have`	Bulk "what am I missing".
`POST`	`/api/v1/check`	Scan an uploaded archive and answer with the verdict. Off by default; keeps nothing.
`GET`	`/song/{chart_hash}.sng`	The bytes.

`GET /api/v1/library`


```
{
  "songs": 22,
  "distinct_charts": 22,
  "duplicate_packages": 0,
  "built_at": "2026-09-05T20:11:05Z",
  "problems": [],
  "sort_attributes": ["name", "artist", "album", "artist_album", "genre", "subgenre",
                      "year", "charter", "playlist", "source", "song_length", "date_added"]
}
```


`songs` counts packages; `distinct_charts` counts songs as YARG identifies them, and the two differ whenever two packages share a chart.

`problems` **is the important field.** It names every directory or archive the scan could not read. A library that quietly indexes 9,000 of 10,000 songs is indistinguishable from one that has 9,000 songs, and an operator has no way to find out which — so failures are surfaced here rather than logged once at start and forgotten.

`GET /api/v1/features`

Every optional capability, whether it is on, and **which of the three configuration sources decided it**.

```
{
  "features": [
    {
      "name": "browse_ui",
      "enabled": true,
      "default": true,
      "source": "default",
      "description": "Phone-friendly page listing the library.",
      "endpoint": "GET /",
      "enable_with": "--browse-ui / browse_ui = yes"
    },
    {
      "name": "check_uploads",
      "enabled": false,
      "default": false,
      "source": "default",
      "description": "Scan an uploaded archive and answer with the verdict, keeping nothing.",
      "endpoint": "POST /api/v1/check",
      "enable_with": "--check-uploads / check_uploads = yes"
    }
  ]
}
```


`source` is `default`, `file` or `flag`. It is the field that earns this endpoint: "*check_uploads is off*" is a fact, while "*check_uploads is off and nothing you wrote mentions it*" tells an operator their config file was never read — the failure that otherwise costs an afternoon, because a file the server never opened produces a server that behaves exactly as if the file did not exist. The startup log now names the file it actually loaded for the same reason.

`enable_with` is present whether the feature is on or off. The reader who needs it is the one looking at a feature that is **off**.

What is deliberately not here: the listen address, the library path and the data path. Those are settings, not capabilities. This endpoint is unauthenticated like the rest, and while which capabilities exist is not a secret — a caller can already tell by fetching `/` or by watching `POST /api/v1/check` answer 404 — a server's filesystem layout is nobody's business on a LAN. That omission is load-bearing rather than an oversight to tidy up later by "just returning the config", and a test asserts it.

`features` is always an array, never `null`. The response also carries `writable` — whether THIS caller may change a feature, not merely whether writing is switched on — plus `writable_reason` when it is false. A page that offered switches to a caller the server will refuse would turn a deliberate configuration choice into what looks like a broken button.

This is the read half of the config menu in `ROADMAP.md` phase 4. A menu has to render the current state, and where that state came from, before anything can sensibly change it. **Nothing here writes**; changing a setting is still a config file or a flag and a restart.

`PUT /api/v1/features/{name}`

Turns one capability on or off, immediately, with no restart. Body:

```
{ "enabled": true }
```

```
{
  "feature": {
    "name": "check_uploads",
    "enabled": true,
    "default": false,
    "source": "default",
    "description": "Scan an uploaded archive and answer with the verdict, keeping nothing.",
    "endpoint": "POST /api/v1/check",
    "enable_with": "--check-uploads / check_uploads = yes"
  },
  "persisted": true,
  "config_file": "yarg-song-server.conf",
  "make_permanent": "check_uploads = yes"
}
```

The change is written back to the config file the server read, so it survives a restart. `persisted` says whether that worked and `config_file` names the file; when it did not, `persist_error` says why and `make_permanent` is the line to add by hand.

A failed save is not a failed toggle. The runtime change stands either way and the response reports the two halves separately — conflating them would send somebody looking for a bug that is not there. The server never *creates* a config file: settings written into whatever directory it happens to be running from would land somewhere nobody would think to look, so if no config file was read, `persisted` is false and the response says so.

Only one line moves. The last **uncommented** line for that key is rewritten — last one wins in this format, so rewriting the first would leave a later line silently overriding it — and when there is none, the setting is appended with a dated comment, leaving the commented template intact. The value is parsed back before anything is written, because persisting a line that stops the server starting next time is the worst outcome a convenience feature can have. The write is atomic through a temp file in the same directory, and keeps the file's permissions and line endings.

Who may call it

Controlled by `config_writes`, which is `off` **by default**:

`config_writes`	Result
`off`	The route is not there. **404**, byte-identical to any unknown path.
`local`	Only a caller from this machine. Anyone else gets **403**.
`lan`	Any caller that can reach the port.

It is not a yes/no because "yes" would have had to mean `lan`, and on a home network that is every device including the ones nobody is thinking about. This server has no authentication, so `lan` means exactly what it says and is a choice an operator should make deliberately.

Locality is read from the socket's own `RemoteAddr`, never from `X-Forwarded-For` or `X-Real-IP`. Those headers are supplied by the caller; trusting one would let anybody on the network claim to be local by typing a line, turning `local` into a suggestion. This server is documented as not belonging behind a reverse proxy, so there is no legitimate case where the socket lies. A test asserts the spoof fails.

What can be changed

Only capabilities: `browse_ui` and `check_uploads`. Everything else answers 404 — paths, the listen address and cache sizes are settings rather than features, and the registry has never carried them.

`config_writes` **itself is not writable**, and answers 403 with that reason. A write surface that can widen its own access has no bound at all: one call turning `local` into `lan` and the "only from this machine" promise is gone, made by whoever was already inside it. Who may configure this server stays a decision made at the machine, in the file.

A disabled feature is absent, not forbidden

Every optional capability answers **404** when it is off, and that 404 is byte-for-byte the one Go's mux sends for a path that does not exist — same status, same body, same content-type. A 403 would tell the caller the feature exists, which is a different fact than the one they asked for.

This used to be true by construction: the route was simply never registered. A feature that can be switched on at runtime cannot work that way, because a route that does not exist cannot be switched on, so the routes are now always registered and the handlers answer instead. The observable contract is unchanged and a test compares the two responses rather than trusting that it is.

Errors

Status	When
400	Body is not JSON, or `enabled` is missing. Absent and `false` are different intents, and a missing field must not silently disable something.
403	The caller is not permitted, or the name is `config_writes`.
404	Writing is off, or the name is not a capability.

`GET /api/v1/songs`

Parameter	Default	Meaning
`q`	—	Free text. Matched against name, artist, album, genre, subgenre, charter, source and playlist.
`sort`	`name`	One of the twelve attributes above. Anything else is a **400**.
`order`	`asc`	`asc` or `desc`. Anything else is a 400.

Parameter	Default	Meaning
`limit`	50	Capped at 500.
`offset`	0	Past the end is an empty page, not an error.

```
{ "total": 22, "offset": 0, "limit": 50, "sort": "artist", "order": "asc",
  "query": "", "songs": [ /* catalog entries */ ] }
```

An unrecognised `sort` is refused rather than silently replaced with the default. A client that asked for one order and was handed another without being told has no way to notice.

On ordering. Results come back in the order the *client* would show them: values are normalised exactly as YARG.Core's `SortString` does — leading article dropped, diacritics folded, rich-text markup stripped — and the tie-breakers are upstream's own comparer chains. See `internal/sortkey` and ADR-002 for what is reproduced and for the two places this server deliberately differs.

On matching. Only the *folding* is the client's, so `q=bjork` finds "Björk" and `q=the beatles` finds "The Beatles". The matching itself — a substring test across those eight attributes — is this server's own. Upstream's search logic has not been read and no parity is claimed for it. A query does not match across a field boundary: `q=blur blur` finds nothing even when the artist and the album are both "Blur".

`GET /api/v1/songs/{chart_hash}`

Returns a **list**, because song identity is deliberately many-to-one:

```
{ "chart_hash": "0856db78…", "count": 2, "songs": [ /* … */ ] }
```

Two packages with the same chart and different audio are the same song to YARG — its own cache is `hash -> List<SongEntry>`. Returning only the first would hide the other from every client that asked.

`POST /api/v1/have`

The client sends the chart hashes it already holds; the answer is everything in the library that was not in that list. One round trip, whatever the size of either side.

```
{ "chart_hashes": ["0856db78…", "b434fc60…"] }
```

```
{ "library_total": 22, "missing": ["2db04926…", "…"], "missing_count": 20 }
```

Hashes are compared case-insensitively and surrounding whitespace is ignored, because a client assembling a list from its own cache should not be punished for either. `missing` is sorted, so the same question twice gives the same bytes.

Chart hash, not package hash, on purpose: a client that has the chart can play the song, and re-sending it a package that differs only in album art is bandwidth spent on nothing.

An unknown field in the body is a **400** rather than being ignored — a client that sent `chart_hash` for `chart_hashes` would otherwise be told, plausibly and wrongly, that it is missing the entire library.

`GET /song/{chart_hash}.sng`

The package bytes, always as `.sng`. A song stored as anything else — a loose folder, a `.zip`, a `.7z` or a `.rar` — is packed on demand and the archive is cached; packing copies the chart byte for byte, so the song's identity is unchanged by it. The three shapes pack to the same bytes, so re-zipping a library does not change what a client downloads.

- `ETag` is the package hash — a hash of the content, so it is the same on every server holding the same package and survives a rescan, a restart and a cache wipe.

It does NOT survive a mirror, and that surprises people. A server whose library is a folder serves a different ETag from a server whose library is the `.sng` that was synced out of it, because the folder carries `song.ini` as a file while the packed `.sng` carries that metadata in its header. Measured on the 23-case corpus: **22 of 23 package hashes change across pack → sync → rescan.** The one that does not is `17-no-song-ini`, which has no `song.ini` to move.

So `ETag` answers "are these the same package", not "are these the same song". **The identity that survives a mirror is the chart hash**, which is what `/api/v1/have` and the sync client compare. Do not write a tool that decides two libraries differ because their ETags do. - `Range` is supported, so an interrupted download resumes rather than starting again. - **300 Multiple Choices** when the chart hash is shared by several packages. The response lists them; `?package=<package_hash>` names one. The server does not choose, because choosing would hand different clients different audio for the same request. - **404** when the hash is unknown, and also when the index points at a file that has since moved — with a message saying to rescan, because that is the fix.

`POST /api/v1/check`

Scans an uploaded archive and answers with the verdict. **Keeps nothing.**

Off by default — `check_uploads = yes`, or `--check-uploads`. When it is off the route is not registered at all, so a server without it answers **404** rather than 403: a 403 would tell the caller the feature exists.

```
curl -s --data-binary @Song.sng \
  "http://pi.local:8080/api/v1/check?name=Song.sng" | jq
```

`?name=` is **required**, and only its EXTENSION is used — to pick the reader, exactly as a library walk picks it from the filename on disk. The name never reaches the filesystem; the body is staged under `<data>/check/` with a name the server chooses, scanned, and deleted before the response is written.

Answer	When
200 with `"accepted": true`	the scanner read it; `song` carries the full metadata, parts and issues, and `known` says whether that chart is already in the library

Answer	When
200 with `"accepted": false` and a `reason`	it is not a song this server would take. **A refusal is not an error**: the request succeeded and the answer was no, and a client has to be able to tell that from "the server broke"
400	no `?name=`
413	body over `check_max_bytes` (default 128 MiB)
415	an extension this server does not read; send `.sng`, `.zip`, `.7z` or `.rar`

A Rock Band console package is refused from the **name**, before a byte of the body is read, with the same reason a library scan gives — nobody should have to upload two gigabytes to be told this server will never read one.

The verdict comes from `scan.ScanFile`, the same function `WalkLibrary` calls for a file on disk. That is deliberate and load-bearing: two implementations of "what is this file" would agree for a while and then quietly stop, and the disagreement would surface as "*the checker said it was fine and the library dropped it*". See ADR-005.

What is not here yet

Storing an upload (ADR-005 increment 2) and authentication. See `docs/ROADMAP.md`. The server is read-only with respect to your library and unauthenticated: run it on a LAN, not on the internet.

Settings are a flag or a `key = value` line in `./yarg-song-server.conf`, same name for both, flag wins; `--write-config` prints a commented example. An unknown setting in that file is an error rather than a warning, on the same reasoning as the unknown-field rule on `POST /api/v1/have`.

`GET /`

A phone-friendly page listing the library: search, the same twelve sort attributes the API offers, and a per-song panel with metadata, parts, any issues the scanner flagged, and a download link. It is served from the binary — no CDN, no build step, nothing external — because the deployment this is aimed at is a Pi on a LAN at a party, where a page that needs the internet fails exactly when it is wanted.

It reads the sort attributes from `/api/v1/library` rather than hardcoding them, so the page cannot drift from the server that served it.

On by default; `browse_ui = no`, or `--browse-ui=false`, turns it off. It exposes nothing `/api/v1/songs` does not already serve unauthenticated to anyone who can reach the port — the decision that matters is the one in ADR-001, that this server is read-only and must not be exposed publicly.

It is registered as `GET /{$}`, an exact match on the root. A bare `GET /` in Go's ServeMux is a catch-all: it would answer every unmatched path with this page and a 200, turning every 404 documented above into an HTML page that a sync client would try to parse as a `.sng`. `TestBrowsePageOnAndOff` pins that, and fails if the pattern is ever loosened.

It offers a drop zone when — and only when — `check_uploads` is on. `/api/v1/library` reports the capability and the page reads it, for the same reason it reads `sort_attributes`: a page that guessed would offer a drop zone against a server that answers 404. Files are checked one at a time, because a dropped folder can be hundreds of files and the

target deployment is a Raspberry Pi.

Every field it renders is escaped, and that was checked rather than assumed. The page renders song metadata that came out of `song.ini` files inside uploaded archives — content the server does not author and cannot vouch for — so a stored-XSS review was done on 2026-09-08. `card()` passes every string through `esc()` (`&<>"'`) and builds the download link with `encodeURIComponent`. Exactly three interpolations bypass `esc` , and each is safe for a reason that does not depend on the value being well-formed:

Site	Why it is not a hole
`"<summary>" + n + " items"`	`n` is `problems.length` , a number.
`"api/v1/songs?" + p.toString()`	`URLSearchParams.toString()` percent-encodes.
`'<b>' + p.intensity + '</b>'`	`Parts.Intensity` is a Go `int8` , so it marshals as a JSON number and cannot carry a string.

The drop zone's verdict is the same data one step earlier and gets the same treatment: every field through `esc()` , the refusal reason assigned as `textContent` , and each field named individually by a test rather than counting `esc(` calls. Driven in a real browser with a file named `<img src=x onerror=…>.sng` : **0 elements injected.**

Recorded as a **non-finding** so it is not re-investigated: the escaping is correct today, and the third row is the one to re-check if `intensity` ever stops being an integer.

What it deliberately does not have is a queue. Upstream's open request #860 asks for search *and queueing* from a phone while YARG is running; queueing needs something inside the running game to read a queue, which is Phase 3 work. A button that cannot work is worse than no button, so there is not one.
